

Top 25 DSA Algorithms Every Developer Should Know

Master these 25 foundational algorithms. They cover **every major category** of DSA — sorting, graphs, strings, dynamic programming, and more. Nearly every hard interview problem reduces to one or a combination of these.

Legend

- ● Sorting · ● Graph · ● String · ● Dynamic Programming · ● Array/Pointer · ● Tree/Data Structure
- Difficulty: ★ Foundational · ★★ Intermediate · ★★★ Advanced
- TC = Time Complexity · SC = Space Complexity

The Algorithms

#	Algorithm	Category	TC	SC	Key Idea	Must-Know LeetCode Problem
1	Binary Search	● Array	$O(\log n)$	$O(1)$	Halve the search space on a sorted or monotonic domain; also works on "search on answer space"	Binary Search #704 · Search in Rotated Sorted Array #33
2	Merge Sort	● Sorting	$O(n \log n)$	$O(n)$	Divide array in half, sort each half, merge — stable sort, foundational divide & conquer	Sort an Array #912 · Count of Smaller Numbers After Self #315
3	Quick Sort	● Sorting	$O(n \log n)$ avg / $O(n^2)$ worst	$O(\log n)$	Pick a pivot, partition array around it, recurse — fastest in practice with good pivot selection	Sort an Array #912 · Kth Largest Element in an Array #215
4	Heap Sort	● Sorting	$O(n \log n)$	$O(1)$	Build a max-heap, repeatedly extract max — in-place; also core to Top-K and priority queue problems	Kth Largest Element in a Stream #703 · Find Median from Data Stream #295
5	Counting Sort / Radix Sort	● Sorting	$O(n + k)$	$O(k)$	Non-comparison sort for integers in a known range; bypasses the $O(n \log n)$ lower bound	Sort Colors #75 · Maximum Gap #164
6	Depth-First Search (DFS)	● Graph	$O(V + E)$	$O(V)$	Explore as deep as possible before backtracking; detects cycles, finds connected components, solves mazes	Number of Islands #200 · Course Schedule #207
7	Breadth-First Search (BFS)	● Graph	$O(V + E)$	$O(V)$	Level-by-level expansion; finds the shortest path in an unweighted graph or grid	Word Ladder #127 · Rotting Oranges #994
8	Dijkstra's Algorithm	● Graph	$O((V + E) \log V)$	$O(V)$	Greedy shortest path in a weighted graph with non-negative edges ; uses a min-heap	Network Delay Time #743 · Path with Minimum Effort #1631
9	Bellman-Ford Algorithm	● Graph	$O(V \times E)$	$O(V)$	Relaxes all edges $V-1$ times; handles negative weights and detects negative cycles	Cheapest Flights Within K Stops #787 · Negative Cycle (classic)
10	Floyd-Warshall	● Graph	$O(V^3)$	$O(V^2)$	DP-based all-pairs shortest path ; $O(V^3)$ but simple — use when V is small (≤ 500)	Find the City With the Smallest Number of Neighbors #1334

11	Topological Sort (Kahn's BFS)	 Graph	$O(V + E)$	$O(V)$	Order nodes of a DAG such that every edge points forward; use in-degree counting for iterative version	Course Schedule II #210 · Alien Dictionary #269
12	Kruskal's / Prim's (MST)	 Graph	$O(E \log E)$	$O(V)$	Build the Minimum Spanning Tree ; Kruskal uses Union-Find + sorted edges, Prim uses a min-heap	Min Cost to Connect All Points #1584 · Connecting Cities With Minimum Cost #1135
13	Union-Find (DSU)	 Data Structure	$O(\alpha(n)) \approx O(1)$	$O(n)$	Tracks disjoint sets with path compression + union by rank; cycle detection in $O(1)$ per query	Redundant Connection #684 · Accounts Merge #721
14	Kadane's Algorithm	 Array	$O(n)$	$O(1)$	Maximum subarray sum via a single pass — the key insight: discard any prefix with negative sum	Maximum Subarray #53 · Maximum Sum Circular Subarray #918
15	Two Pointers	 Array	$O(n)$	$O(1)$	Squeeze from both ends of a sorted array/string to find pairs/triplets in linear time	Two Sum II #167 · 3Sum #15
16	Sliding Window	 Array/String	$O(n)$	$O(1)$ – $O(k)$	Expand/shrink a window over contiguous elements to satisfy a constraint without re-scanning	Minimum Window Substring #76 · Sliding Window Maximum #239
17	Floyd's Cycle Detection (Fast & Slow)	 Linked List	$O(n)$	$O(1)$	Two pointers at different speeds detect a cycle in $O(1)$ space — no visited set needed	Linked List Cycle II #142 · Find the Duplicate Number #287
18	KMP (Knuth-Morris-Pratt)	 String	$O(n + m)$	$O(m)$	Build a failure function to skip redundant comparisons — pattern matching without backtracking	Find the Index of the First Occurrence in a String #28 · Shortest Palindrome #214
19	Rabin-Karp (Rolling Hash)	 String	$O(n + m)$ avg	$O(1)$	Use a rolling hash to compare substrings in $O(1)$ — ideal for multi-pattern search	Repeated String Match #686 · Longest Duplicate Substring #1044
20	Dynamic Programming — Knapsack	 DP	$O(n \times W)$	$O(W)$	Classic DP template: pick-or-skip decisions with a capacity constraint — basis for 50%+ of DP problems	Partition Equal Subset Sum #416 · Target Sum #494
21	Dynamic Programming — LCS / Edit Distance	 DP	$O(m \times n)$	$O(m \times n)$	Fill a 2D DP table for string alignment; foundation for diff tools, spell checkers, and bioinformatics	Longest Common Subsequence #1143 · Edit Distance #72
22	Dynamic Programming — Interval DP	 DP	$O(n^3)$	$O(n^2)$	Solve problems on all sub-intervals $[i, j]$ by combining results of smaller sub-intervals	Burst Balloons #312 · Strange Printer #664
23	Backtracking	 Search	$O(b^d)$	$O(d)$	Explore decision trees, pruning invalid branches early; generates all valid combinations/permutations	N-Queens #51 · Sudoku Solver #37
24	Greedy (Interval Scheduling / Activity Selection)	 Greedy	$O(n \log n)$	$O(1)$	Make the locally optimal choice at each step; works when greedy choice property + optimal substructure hold	Non-overlapping Intervals #435 · Jump Game II #45

25	Huffman Coding / Greedy Encoding	🔴 Tree	$O(n \log n)$	$O(n)$	Build an optimal prefix-free code using a greedy min-heap merge — basis for all lossless compression	Minimum Cost to Connect Sticks #1167 · Rearrange String k Distance Apart #358
----	----------------------------------	--------	---------------	--------	--	---

Category Breakdown

Category	Algorithms	Why They Matter
🟦 Sorting	Binary Search, Merge Sort, Quick Sort, Heap Sort, Counting Sort	Foundation for $O(n \log n)$ thinking; Quick Select solves Top-K in $O(n)$
🟪 Graph	DFS, BFS, Dijkstra, Bellman-Ford, Floyd-Warshall, Topo Sort, MST, Union-Find	Graphs model almost every real-world problem — networks, dependencies, maps
🟢 Array / Pointer	Kadane's, Two Pointers, Sliding Window, Floyd's Cycle, Backtracking, Greedy	The "bread and butter" of coding interviews — $O(n)$ solutions to array problems
🟠 String	KMP, Rabin-Karp	Core algorithms behind <code>grep</code> , search engines, and DNA sequencing tools
🟡 Dynamic Programming	Knapsack, LCS/Edit Distance, Interval DP	Unlocks hard optimization problems; understand these 3 DP shapes to solve almost all DP
🔴 Tree / Data Structure	Union-Find, Heap Sort, Floyd's Cycle, Huffman	Algorithms tied to specific data structures — learn the structure to understand the algorithm

Complexity Cheat Sheet

Algorithm	Best Case	Average	Worst Case	Space
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$
BFS / DFS	$O(V + E)$	$O(V + E)$	$O(V + E)$	$O(V)$
Dijkstra	$O(E)$	$O((V+E)\log V)$	$O((V+E)\log V)$	$O(V)$
Bellman-Ford	$O(E)$	$O(V \times E)$	$O(V \times E)$	$O(V)$
Floyd-Warshall	$O(V^3)$	$O(V^3)$	$O(V^3)$	$O(V^2)$
Topological Sort	$O(V + E)$	$O(V + E)$	$O(V + E)$	$O(V)$
Kruskal / Prim	$O(E \log E)$	$O(E \log E)$	$O(E \log E)$	$O(V)$
Union-Find (DSU)	$O(\alpha(n))$	$O(\alpha(n))$	$O(\alpha(n))$	$O(n)$
Kadane's	$O(n)$	$O(n)$	$O(n)$	$O(1)$
KMP	$O(n + m)$	$O(n + m)$	$O(n + m)$	$O(m)$
Rabin-Karp	$O(n + m)$	$O(n + m)$	$O(n \times m)$	$O(1)$
DP (Knapsack)	—	$O(n \times W)$	$O(n \times W)$	$O(W)$
DP (LCS)	—	$O(m \times n)$	$O(m \times n)$	$O(m \times n)$
Backtracking	$O(1)$	$O(b^d)$	$O(b^d)$	$O(d)$

Learning Order (8-Week Roadmap)

Week 1	→ Binary Search · Merge Sort · Quick Sort · Heap Sort · Counting Sort Goal: Master $O(n \log n)$ thinking and the divide & conquer paradigm
Week 2	→ BFS · DFS · Kadane's · Two Pointers · Sliding Window · Floyd's Cycle Goal: Core linear-time techniques; cover 60% of easy/medium problems
Week 3	→ Dijkstra · Bellman-Ford · Floyd-Warshall · Topological Sort Goal: Graph shortest-path and ordering – the backbone of graph interviews
Week 4	→ Kruskal's / Prim's · Union-Find Goal: Minimum Spanning Trees and dynamic connectivity
Week 5	→ KMP · Rabin-Karp Goal: String pattern matching – often tested as "implement strstr()"
Week 6	→ DP Knapsack · DP LCS / Edit Distance Goal: The two most common 2D DP shapes – once you see them, you can adapt anything
Week 7	→ DP Interval · Backtracking Goal: Hard DP and exhaustive search with pruning
Week 8	→ Greedy · Huffman Coding · Review all 25 with timed problems Goal: Solidify greedy intuition and integration of all 25 algorithms

When to Use Which Algorithm

Problem Signal	Algorithm to Reach For
"Find X in sorted array" or "minimize/maximize with a monotonic condition"	Binary Search
"Shortest path, unweighted"	BFS
"Shortest path, weighted non-negative"	Dijkstra
"Shortest path with negative edges"	Bellman-Ford
"All-pairs shortest path"	Floyd-Warshall
"Dependency order / cycle in directed graph"	Topological Sort
"Connect all nodes with minimum cost"	Kruskal / Prim
"Dynamic connectivity, group membership"	Union-Find
"Maximum subarray / circular subarray"	Kadane's
"Find pair/triplet in sorted array"	Two Pointers
"Subarray with constraint (sum, unique chars)"	Sliding Window
"Cycle in linked list / duplicate in array with $O(1)$ space"	Floyd's Cycle Detection
"Pattern match in a string"	KMP / Rabin-Karp
"Pick or skip items with a capacity"	DP — Knapsack
"Align / transform two strings"	DP — LCS / Edit Distance
"Enumerate all valid combinations"	Backtracking
"Schedule tasks, non-overlapping intervals"	Greedy
"Minimum cost to merge / combine elements"	Huffman / Greedy Heap

Key Insight Per Algorithm

1. **Binary Search** — Works on any *monotonic* function, not just sorted arrays. If $f(x)$ is monotone, binary search the answer.
2. **Merge Sort** — The merge step solves more than just sorting: inversion count, external sort, and K-way merge all use it.
3. **Quick Sort** — `partition()` alone gives you **Quick Select** for $O(n)$ average Kth element without full sort.
4. **Heap Sort** — A max-heap is just an array with the invariant $arr[i] \geq arr[2i+1]$ and $arr[i] \geq arr[2i+2]$.
5. **Counting Sort** — When integers are in range $[0, k]$, you can sort in $O(n + k)$. For $k \ll n$, this is linear.
6. **DFS** — DFS on a graph equals a recursive/stack-based traversal. Pre/In/Post-order tree traversals are all DFS.
7. **BFS** — BFS always finds the shortest path in an unweighted graph. The frontier expands one "level" at a time.
8. **Dijkstra** — A greedy BFS with a min-heap. The key insight: once a node is popped from the heap, its distance is final.
9. **Bellman-Ford** — Relax all edges $V-1$ times. A V th relaxation means a negative cycle exists.
10. **Floyd-Warshall** — $dp[i][j] = \min(dp[i][j], dp[i][k] + dp[k][j])$ over all intermediate nodes k .
11. **Topological Sort** — Only valid for DAGs. Kahn's BFS (in-degree tracking) is easier to implement than DFS-based.
12. **Kruskal** — Sort edges by weight, add edge if it doesn't form a cycle (check via Union-Find).
13. **Union-Find** — Path compression + union by rank makes each query nearly $O(1)$. The inverse Ackermann $\alpha(n) \leq 4$ for all practical n .
14. **Kadane's** — $maxEndingHere = \max(num, maxEndingHere + num)$. If the running sum goes negative, restart.
15. **Two Pointers** — Works when the array is sorted and you can decide to move left or right pointer based on comparison.
16. **Sliding Window** — Use a hash map / counter to track window state. Shrink the window when the constraint is violated.
17. **Floyd's Cycle** — Fast pointer moves $2x$, slow $1x$. If they meet, there's a cycle. To find the entry, reset one pointer to head.
18. **KMP** — Build the `lps[]` (Longest Proper Prefix which is also Suffix) array in $O(m)$, then use it to avoid re-scanning.
19. **Rabin-Karp** — Rolling hash: subtract leading character, multiply, add new character. Handles multiple pattern searches well.
20. **Knapsack DP** — Iterate items outer, capacity inner. $dp[w] = \max(dp[w], dp[w - weight] + value)$ for 0/1 knapsack.
21. **LCS / Edit Distance** — $dp[i][j]$ depends on $dp[i-1][j-1]$, $dp[i-1][j]$, $dp[i][j-1]$. Diagonal = match, row/col = gap.
22. **Interval DP** — Try all split points k : $dp[i][j] = \min/\max \text{ over } k \text{ in } (i..j) \text{ of } (dp[i][k] + dp[k+1][j] + cost(i, j))$.
23. **Backtracking** — State $\rightarrow$ make choice $\rightarrow$ recurse $\rightarrow$ undo choice. Always ask: "what is the choice at each step?"
24. **Greedy** — Prove correctness via exchange argument: swapping a greedy choice for any other choice can only make things worse or equal.
25. **Huffman** — Greedily merge the two lowest-frequency nodes using a min-heap. Result is an optimal prefix-free code.